Simon Schmidt
9.16.2026
ASU ID: 1235114116

Code for part 1, Least Squares implementation:

```
21
22  # Lease Squares section
23
24  bias = np.ones((422, 1))
25  design_X = np.hstack((bias, diabetes_X_train))
26  # print("Design:", design_X.shape)
27
28  diabetes_y_train_col = diabetes_y_train.reshape(-1, 1)
29  # print("y_train_col:", diabetes_y_train_col.shape)
30
31  design_dot_inverse = (np.linalg.inv(design_X.T @ design_X))
32  theta = design_dot_inverse @ design_X.T @ diabetes_y_train_col
33
```

Least Squares output with Scikit comparison:

```
❯ python3 hw1.py
The shape of the input features: (442, 10)
The shape of the output variables: (442,)
(442, 1)
Training input variable shape:  (422, 1)
Test input variable shape:  (20, 1)

Least Squares bias and theta 1: [[152.91886183]
 [938.23786125]]
scikit bias: 152.91886182616113
scikit theta_1: [938.23786125]
 ~/ASU/fall_26/cse_475/475_assignments git master !2 ?1 (
```

Code for part 2, Gradient descent implementation:

```
37  # Gradient Descent Section
38
39  rand_theta = np.zeros((2, 1))
40  learn_rate = 0.03
41
42  for t in range(100000):
43      loss = np.sum((design_X @ rand_theta - diabetes_y_train_col) ** 2)
44      loss *= 1 / design_X.shape[0]
45      grad_theta_L = (design_X @ rand_theta - diabetes_y_train_col)
46      grad_theta_L = 2 / design_X.shape[0] * (design_X.T @ grad_theta_L)
47      rand_theta = rand_theta - learn_rate * grad_theta_L
48
```

Gradient descent output with Scikit comparison:

```
❯ python3 hw1.py
The shape of the input features: (442, 10)
The shape of the output variables: (442,)
(442, 1)
Training input variable shape:  (422, 1)
Test input variable shape:  (20, 1)

Gradient Descent bias and theta 1: [[152.91886238]
 [938.2366844 ]]
scikit bias: 152.91886182616113
scikit theta_1: [938.23786125]
~/ASU/fall_26/cse_475/475_assignments git master !2 ?1 ❮
```

hw1.py file as a whole:

```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets
from sklearn.linear_model import LinearRegression
plt.show()

diabetes_X, diabetes_y, = datasets.load_diabetes(return_X_y=True)
print('The shape of the input features:', diabetes_X.shape)
print('The shape of the output variables:', diabetes_y.shape)

diabetes_X_one = diabetes_X[:, np.newaxis, 2]
print(diabetes_X_one.shape)

diabetes_X_train = diabetes_X_one[:-20]
diabetes_X_test = diabetes_X_one[-20:]
diabetes_y_train = diabetes_y[:-20]
diabetes_y_test = diabetes_y[-20:]

print("Training input variable shape: ", diabetes_X_train.shape)
print("Test input variable shape: ", diabetes_X_test.shape)

# Lease Squares section

bias = np.ones((422, 1))
design_X = np.hstack((bias, diabetes_X_train))
print("Design:", design_X.shape)

diabetes_y_train_col = diabetes_y_train.reshape(-1, 1)
print("y_train_col:", diabetes_y_train_col.shape)

design_dot_inverse = (np.linalg.inv(design_X.T @ design_X))
theta = design_dot_inverse @ design_X.T @ diabetes_y_train_col


# Gradient Descent Section

rand_theta = np.zeros((2, 1))
learn_rate = 0.03

for t in range(100000):
    loss = np.sum((design_X @ rand_theta - diabetes_y_train_col) ** 2)
    loss *= 1 / design_X.shape[0]
    grad_theta_L = (design_X @ rand_theta - diabetes_y_train_col)
    grad_theta_L = 2 / design_X.shape[0] * (design_X.T @ grad_theta_L)
    rand_theta = rand_theta - learn_rate * grad_theta_L

model = LinearRegression()
model.fit(diabetes_X_train, diabetes_y_train)
print("Least Squares bias and theta 1:", theta)
print("Gradient Descent bias and theta 1:", rand_theta)
print("scikit bias:", model.intercept_)
print("scikit theta_1:", model.coef_)
```

Part 3, regularized loss derivation:

